



Java - Data Structures

[Visit our website](#)

Introduction

This task introduces the array, a fundamental data structure in Java used to efficiently store collections of elements of the same type. It covers single-dimensional and multidimensional arrays, explaining how to declare, create, initialise, and work with them in Java. Arrays are essential for fast data access and memory efficiency, making them foundational for programming tasks in Java.

WHAT IS AN ARRAY?

The array is a data structure, provided by Java, which stores a **fixed number of elements of the same type**. An array is used to store a collection of data. However, it is more useful to think of an array as a collection of elements of the same type.

DECLARE AN ARRAY

To use an array in a program, a variable to reference the array must be declared, and the array's element type must be specified. The array can have any element type. All elements that are in the array will have the same data type. To declare a variable, use the following syntax:

```
elementType[] arrayVariable;
```

The following example declares a variable **numArray** that references an array that stores elements of the type **double**.

```
double[] numArray;
```

CREATE AN ARRAY

After declaring an array variable you can create an array using the **new** operator. Technically speaking, the **new** keyword dynamically (i.e. at run time) allocates memory for the array and returns a reference to that memory. The syntax is as follows:

```
elementType[] arrayVariable = new elementType[arraySize];
```

The following example declares an array variable called **numArray**, and creates an array of

5 double elements and assigns the reference to the array to **numArray** variable.

```
double[] numArray = new double[5];
```

The array size must be given when space for an array is allocated. The size cannot be changed after an array is created. To find the size of an array, use **arrayVariable.length**. For example:

```
System.out.println(numArray.length); // Prints out 5
```

Numeric data types are assigned the default value 0, **char** types are assigned the value `\u0000` and boolean types are assigned the value **false** when an array is created. If you have an **Object** array, the default value will be null. We will cover **Object** and **null** in later lessons.

ARRAY INDICES

To access array elements you use an index. Array indices range from 0 to **arrayVariable.length - 1**. For example, the array **numArray** created above, holds five elements with indices from 0 to 4. You can represent each element in the array using the following syntax:

```
arrayVariable[index];
```

The index needs to be of **int** type. For example, for a non-empty array **numArray**, **numArray[0]** represents the first element in the array **numArray**. **numArray[1]** represents the second element in the array **numArray**. And **numArray[numArray.length - 1]** represents the last element in the array **numArray**.

If you use an index which is negative or larger than **numArray.length - 1** to access the element of the array, you will get an **ArrayIndexOutOfBoundsException** exception, because the index value is illegal and there is no such element in the array.

INITIALISE AN ARRAY

To assign values to the elements we will use the syntax:

```
arrayVariable[index] = value;
```

The example below initialises the array referenced by the variable `numArray`:

```
numArray[0] = 23.6;  
numArray[1] = 45.12;  
numArray[2] = 8.4;  
numArray[3] = 77.7;  
numArray[4] = 1.34;
```

You can use shorthand notation in Java, known as the array initialiser, which combines the declaration, creation and initialisation of an array in one statement.

The syntax for an array initialiser is as follows:

```
elementType[] arrayVariable = {value1, value2, ..., valueN};
```

For example, the following array initialiser statement is equivalent to the example above in which each element was assigned a value individually:

```
double[] numArray = {23.6, 45.12, 8.4, 7.77, 1.34};
```

TWO-DIMENSIONAL OR HIGHER-DIMENSIONAL ARRAYS

Previously, we have covered simple single-dimensional arrays. However, arrays are not limited to one dimension. The dimension of an array can be two or even higher. Very often, you can use a two-dimensional array to store data in a table or matrix.

DECLARE AND CREATING A TWO-DIMENSIONAL ARRAY

The syntax for creating a two-dimensional array is as follows:

```
elementType[][] arrayVariable;
```

The following example demonstrates how to declare and create a two-dimensional array of 5-by-5 int values.

```
int[][] table;  
table = new int[5][5];
```

Each number between the square brackets represents the number of rows and columns in a table. The first index represents the rows and the second represents the columns. Each index begins with a 0, as with a one-dimensional array.

ASSIGNING VALUES TO ELEMENTS IN A TWO-DIMENSIONAL ARRAY

Like a single-dimensional array, use the array indices to access elements in a two-dimensional array. However, unlike a one-dimensional array, a two-dimensional array contains two sets of indices.

The example below shows how to assign the number 4 to the element in the second row and first column:

```
table[1][0] = 4;
```

The array initialiser can also be used to declare, create and initialise a two-dimensional array.

The following example creates an array and initialises it with values.

```
int[][] array = { {1, 2, 3}, {4, 5, 6}, {7, 8, 9}, {10, 11, 12} };
```

This is equivalent to the following:

```
int[][] array = new int[4][3];  
array[0][0] = 1;   array[0][1] = 2;   array[0][2] = 3;  
array[1][0] = 4;   array[1][1] = 5;   array[1][2] = 6;  
array[2][0] = 7;   array[2][1] = 8;   array[2][2] = 9;  
array[3][0] = 10;  array[3][1] = 11;  array[3][2] = 12;
```



Take note:

Ragged Arrays

In a two-dimensional array, each row is itself an array. Therefore, the rows can have different lengths. Arrays with rows of varying lengths are known as ragged arrays.

Here is an example of a ragged array:

```
int[][] triArray = { {1, 2, 3, 4},  
                    {5, 6, 7},  
                    {8, 9},  
                    {10} };
```

If you know the sizes of a ragged array, but do not know the values of the elements you can still create one by using the following syntax:

```
int[][] triArray = new int[4][];  
triArray[0] = new int[4];  
triArray[1] = new int[3];  
triArray[2] = new int[2];  
triArray[3] = new int[1];
```

Note that when using `new int [4] [ ]` to create a new array, the first index (number of rows) must always be specified

PROCESSING AN ARRAY

The easiest way to process the elements of an array is by using a **for** loop. A **for** loop is the natural choice when processing arrays because all of the elements in an array have the same data type and are processed in the same way. Since the size of the array is known, it makes sense to use a **for** loop rather than another repetition structure. Suppose you have created the following array:

```
int[] myArray = new int[6];
```

You can use a **for** loop to initialise the array with values entered by a user as follows:

```
java.util.Scanner input = new java.util.Scanner(System.in);  
System.out.print("Please enter " + myArray.length + " values");  
for (int i = 0; i < myArray.length; i++) {  
    myArray[i] = input.nextDouble();  
}  
  
for (int i = 0; i < myArray.length; i++) {  
    System.out.print(myArray[i] + " ");  
}
```

A sample output of this code is:

```
Please enter 6 values
1
1
2
3
5
8
1 1 2 3 5 8
```

The code first prints out a line in the console to ask you to enter 6 values. 6 comes from the size (length) of the array **myArray**. Then the code enters the first **for**-loop, which iterates 6 times. In each iteration, the Scanner object input will wait for you to enter a value in the console. The value you entered will be assigned to the *ith* element of **myArray**. Please be aware that Java is 0-index based. This means the first element of an array starts with index 0, instead of 1.

After 6 values are entered, the first **for**-loop exits and the code enters the second **for**-loop, which prints out each element of the array **myArray**.

As you can see, it would be quite difficult to process each of the array's elements individually, especially if the array is large. The for loop is a lifesaver when it comes to performing repetitive operations on elements in an array.

PROCESSING A TWO-DIMENSIONAL ARRAY

To process a two-dimensional array is similar to processing a one-dimensional array. We can use a nested **for**-loop to traverse a two-dimensional array. For example:

```
int[][] twoDimensionArray = { {1, 2, 3}, {4, 5, 6}, {7, 8, 9}, {10, 11, 12} };
for (int i = 0; i < twoDimensionArray.length; i++) {
    for (int j = 0; j < twoDimensionArray[i].length; j++) {
        System.out.print(twoDimensionArray[i][j] + " ");
    }
    System.out.println();
}
```

It prints out:

```
1 2 3
```

```
4 5 6
7 8 9
10 11 12
```

The outer **for**-loop iterates over each row. The number of rows is `twoDimensionArray.length`, which is 4. The inner **for**-loop iterates over each column for the current row traversed. The number of columns is `twoDimensionArray[i].length`, which is 3 for all the rows in this example.

ARRAYLIST

You will be learning about **ArrayList** in more detail in a later task, but for now, you can see ArrayList as an array in which its length is malleable — you can insert and remove elements as you like.

ArrayList is not technically an array; it's a Java Collection type. Java's **Collections API** provides interfaces for common data structures.

A List is an ordered Collection where each element has a specific position, similar to an array but with a flexible size. Unlike arrays, an ArrayList can dynamically resize, allowing you to add or remove elements without specifying an initial size.

In contrast, an array's length is fixed once set and cannot be changed.

USING ARRAYLIST

You can create and initialise an ArrayList as follows:

```
import java.util.ArrayList;

ArrayList<String> drivers = new ArrayList<>();
```

This statement creates an instance of ArrayList called drivers. The `<String>` specifies the type of elements stored in the List. This `<>` is a Type parameter which will be covered in the future.

Please also note that you need to explicitly add an import statement **import java.util.ArrayList;**. It is because ArrayList is in the package `java.util`, which Java program does not load by default.

```

drivers.add("Larry");
drivers.add("Peter");
drivers.add("Gabe");
System.out.println(drivers); // [Larry, Peter, Gabe]

drivers.add(0, "Jill"); // Adds Jill at index 0
System.out.println(drivers); // [Jill, Larry, Peter, Gabe]

String firstDriver = drivers.get(0);
System.out.println(firstDriver); // Jill

drivers.remove(1); // Removes item at index 1
System.out.println(drivers); // [Jill, Peter, Gabe]

System.out.println(drivers.size()); // 3

for (int i = 0; i < drivers.size(); i++) {
    if (i > 0) { // Print comma after each driver's name
        System.out.print(", ");
    }
    System.out.print(drivers.get(i));
}

```

The above sample shows some basic usage of ArrayList's methods. You can use the **add(element)** method to add an element to the end of the list (remember, List is an ordered collection). You can also use **add(index, element)** to add the element at the given index. Please be reminded that the index of Java Collection types starts with 0. So adding an element to index 0 means to add the element to the first position.

To get an element from the List at a specific position, you can use **get(index)**. And to remove an element at a specific position, you can use **remove(index)**.

You can use the **size()** method to get the current size of the List. This is the number of elements contained in the List.

The example above shows how to loop through an ArrayList using a for-loop. Inside the for-loop, a counter **i** is initialised as 0 and runs as long as it is smaller than the size of the ArrayList. In this way, the number of iterations of the for-loop is exactly the size of the ArrayList. In each iteration, the code prints out the **i** th element of the ArrayList, using **get()** method.

```

System.out.println(drivers); // [Jill, Peter, Gabe]

```

```
int countOfNamesWithFourLetters = 0;
for (int i = 0; i < drivers.size(); i++) {
    if (drivers.get(i).length() == 4) { // Check that the String length of the
name is equal to 4
        countOfNamesWithFourLetters++; // Increment the counter of names
    }
}
System.out.println("Number of names with four letters: " +
countOfNamesWithFourLetters); // Number of names with four letters: 2
```

Here is another example of finding how many names with 4 letters exist inside the **drivers** ArrayList. The **drivers** ArrayList contains elements [Jill, Peter, Gabe]. The code loops through the ArrayList. In each iteration, it checks the length of each element to be 4. **length()** method of String class can be used to get the length of a String. If the length is 4, the counter variable is incremented. After exiting the for-loop, the value of the counter shows the count of names with 4 letters.

COMMON METHODS OF ARRAYLIST

Common ArrayList methods include:

- *add(element)* - adds the element to the end of the List
- *add(index, element)* - adds the given element at the given index
- *contains(element)* - returns a boolean to show if the given element is in the ArrayList
- *get(index)* - returns the element at the given index
- *indexOf(element)* - returns the index of the given element
- *isEmpty()* - returns a boolean to show if the ArrayList has no elements
- *remove(index)* - removes the element at the given index
- *size()* - returns the number of elements in the ArrayList

You can refer to the [Java SDK 21 API documentation page](#) for more information.

COPYING ARRAYS

One of the common operations on arrays is to copy an array. Sometimes, you may want to create a duplicate array of a given array for further processing, so the processing won't change the values of the original array. Or sometimes, you may want to change the size of an array while maintaining the same elements of a given array. Please be noted that you cannot change the size of an instantiated array. You can only create a new array and copy the elements over to have an array of different sizes.

One of the simplest ways is to use `Arrays.copyOf()` method.

```
String[] array = {"Alex", "Bill", "Cathy", "Deborah"};

String[] copiedArray = Arrays.copyOf(array, 4);
System.out.println(Arrays.toString(copiedArray)); // [Alex, Bill, Cathy, Deborah]

String[] anotherCopiedArray = Arrays.copyOf(array, 6);
System.out.println(Arrays.toString(anotherCopiedArray)); // [Alex, Bill, Cathy, Deborah, null, null]
```

The `Arrays` class contains various methods for manipulating arrays (such as copying, sorting and searching). The `copyOf()` method copies the specified array, truncating or padding with nulls (if necessary) so the copied array has the specified length.

In the first case, `Arrays.copyOf(array, 4)`, the array instance is copied to a new array with a specific size of 4. As the array and the copied array have the same size, the elements of the copied array are the same as shown in the commented print out.

In the next case, `Arrays.copyOf(array, 6)`, the array instance is copied to a new array with a specific size of 6. As the copied array has a size larger by 2, the last 2 elements of the copied array are filled with `null` as shown in the commented print out just above.

For details of `Arrays` class or `copyOf()` method, [please refer to the documentation here](#).

Practical Task 1

Follow these steps:

- Create a new file called **IQ.java**
- Create an array called **testResults** that stores a list of results that 5 people have scored on an IQ test.
- Write the code that will:
 - Ask the user to input 5 integers in the Console and store them in **testResults**.
 - Calculate the average IQ of the five people based on the results in **testResults**.
- Based on the average IQ, decide which description should be assigned.

Use the table below:

IQ score	Description
130+	Very high
120-129	High
110-119	High average
90-109	Average
80-89	Low average
70-79	Borderline
69 and below	Extremely low

Be sure to place files for submission inside your **task folder** and click "Request review" on your dashboard.

Practical Task 2

Follow these steps:

- Create a new file called **MultipleElements.java**
- Write a program that creates a 3 x 4 two-dimensional array with the first 2 multiples of the numbers in the first row.
- Firstly, ask the user to enter four numbers. These will inhabit the first row of your array.
- The next two rows will be the first two multiples of the numbers above them. For example, if the user inputs 1, 2, 3 and 4, the 3-by-4 array will look like this:

```
1  2  3  4
2  4  6  8
3  6  9  12
```

- Print out the array.

Be sure to place files for submission inside your task folder and click "Request review" on your dashboard.

Practical Task 3

Follow these steps:

- Create a new file called **LongWords.java**

- Write a program that determines how many words greater than 4 letters long have been entered and calculate the average length of all the words entered.
- Firstly, ask the user to enter any number of words and store them in an ArrayList. *The user should enter 0 to indicate the end of their input.*
- The program should then determine the number of words longer than 4 letters entered by the user, and print out the result.
- The average length of the words given should then be calculated and displayed.
- For example, the user inputs:

apple

pear

orange

fig

0

The output should be:

2

4.5

Because there are 2 words longer than 4 letters, and the average length = $(5 + 4 + 6 + 3) / 4 = 4.5$

Be sure to place files for submission inside your **task folder** and click "Request review" on your dashboard.



Share your thoughts

HyperionDev strives to provide internationally excellent course content that helps you achieve your learning outcomes.

Do you think we've done a good job or do you think the content of this task, or this course as a whole, can be improved?

Share your thoughts anonymously using this [form](#).
